

Database design

Building and structuring your
own relational database

Carl Nordlund

www.carlnordlund.net



Today's Agenda

From blank page to working database — design, build, populate, query

01

Schema Design

Two worked examples
One-to-many & many-to-many
ER diagrams

02

Creating Tables

Full R workflow
CREATE TABLE
Constraints & data types

03

Populating Data

INSERT · UPDATE · DELETE
dbWriteTable from R
CRUD basics

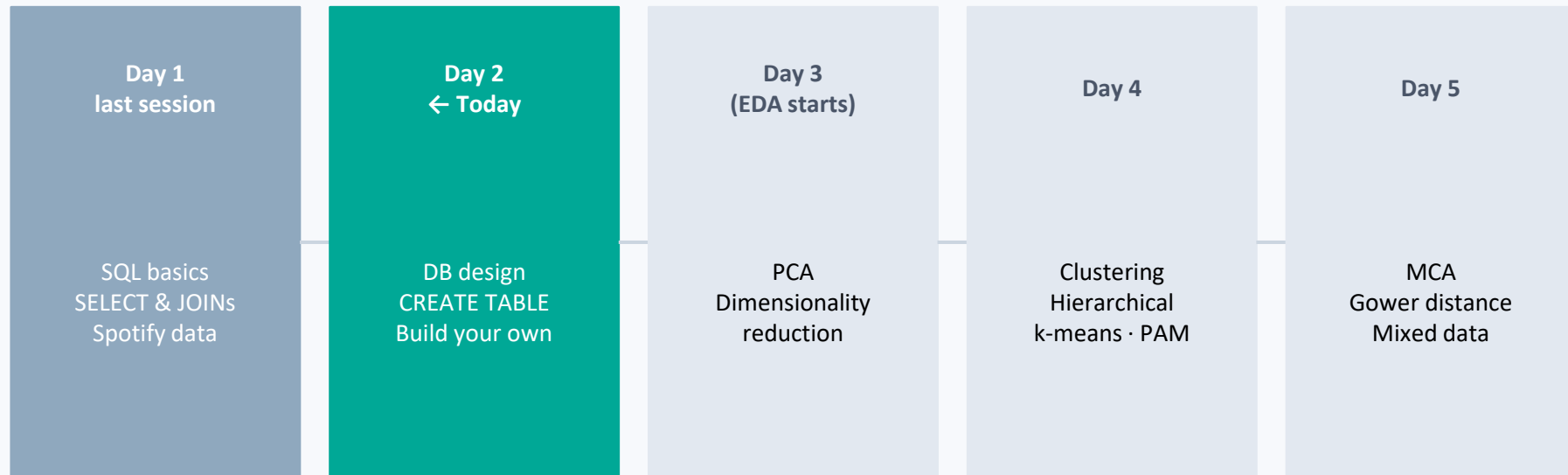
04

Lab

Design your schema
Build & populate
Query your own DB

Where We Are in This Course

Day 2 of 5 — from querying to designing



Today we stop querying someone else's data — and start building our own.

2nd assignment: Exploring a dataset

Instructions

In this 2nd assignment of the course, you will apply the exploratory data techniques that you have learned during the course on a dataset of your choice. You can either pick a dataset from the collection that we will provide (available from 25th May), or you could choose another dataset that you prefer. If you want to pick your own dataset, first check with your teacher (Carl) that it is suitable for this assignment.

Requirements

The assignment must be done individually and has to be submitted as a paper (~7 pages excluding appendix, 1.5 line spaced). You are reasonably free to structure your hand-in report/paper, but it must contain the following parts/aspects:

- Describe (in text) what the dataset is concerned with. Outline an area of inquiry and/or a tentative substantive question (e.g. How do songs cluster together? Are opinions split by party belonging? Are environmental issues a latent variable?). Feel free to include external literature to build a tentative question.
- Use at least two EDA methods that you have learned in this course. A suitable combination is a dimensionality-reduction method followed by clustering, though other combinations might also be acceptable. Check with your teacher if unsure. Note that SQL queries do not constitute an EDA method.
- Justify your choice of method: why these ones? What are their limitations?
- Interpret your results (including dimensions, clusters etc): feel free to reason freely about the results you obtain
- Include a appendix with your code (that must be commented)

Deadline: Friday 12th June 23.59

Format: Approx 7 pages (excluding appendix), PDF format (use your liu username for the document name, for instance: carno10_project2.pdf)

Submit your paper on Lisam (check Submissions section).

Datasets

Datasets will be provided on 25th May (through Lisam). Do note that you may not use the Spotify dataset (music.sqlite) for this assignment. If you want to work with a different dataset, please contact carl.nordlund@liu.se

2nd assignment

Available on Lisam (Documents)

Datasets available next week
25th May (afternoon)

How Do You Store Data From Multiple Sources?

A problem you have already encountered

Twitter / X API

users, tweets,
hashtags, retweets

Statistics portal

municipality tables,
CSV exports

News scraper

articles, authors,
outlets, topics

How do you store this data so it can be combined, updated, and queried together?

The naive answer: one CSV per source, merge in R before each analysis. The problems: duplication, version drift, manual errors etc. A relational database solves this structurally.

Three Core Concepts Before We Design Anything

Entity · Relationship · Normalisation

1

Entity

A thing you want to store information about.

An individual person, an address, a job record, an article, a tweet.

Entities become tables. Each row is one instance.

2

Relationship

How entities connect to each other.

One-to-many: one person, many years of address records.

Many-to-many: many articles, many authors — needs a junction table.

3

Normalisation

Store each fact exactly once.

Ask: will this information ever change independently of the row it sits in? If yes — it belongs in its own table.

These three decisions, i.e. what entities exist, how they relate, and what each table owns, are research design questions, not technical ones.

1

Designing a Schema

Two worked examples: one-to-many, then many-to-many

Two Worked Examples

We build each schema from scratch, starting from a research question

1

Municipality Register Data

Example 1 — focus on one-to-many & temporal data

Linked population register data for all individuals in a municipality. Static demographics, annual employment records, annual address data, geographic hierarchy of neighbourhoods and municipalities.

Key new concept: what is fixed, and what changes over time?

2

Academic Journal Articles

Example 2 — focus on many-to-many & junction tables

Scraped articles from sociology journals: titles, abstracts, years, DOIs, authors, keywords, and citation counts.

Key new concept: when neither side can hold the foreign key, then you need a so-called 'junction table'.

Example 1 — Municipality Register Data

A social-scientific research project: preparing the data pipeline

Research question:

Does growing up in a high-poverty neighbourhood affect employment outcomes in early adulthood, controlling for individual background characteristics?

The data you have been given (different entities):

Population register

All individuals resident in the municipality 2000–2020. Static properties: sex, birth year, birth country.

Employment records

Annual: employment status, sector, income, number of days employed. One row per person per year.

Address register

Annual: which property each person lives at. Includes housing tenure (owner / rental / public).

Geographic data

Properties → neighbourhoods (local areas) → municipalities. Neighbourhood poverty rates from census.

The Key Design Decision — Fixed vs. Time-Varying

Separate what changes from what does not

Fixed → individuals table

- birth_year
- sex
- birth_country
- education_level

Changes annually → separate tables

- residences(person_id, year, neighbourhood_id)
- employment(person_id, year, employed, income)

Why does this matter? The update anomaly:

Without separation:

```
person_id  year  birth_year  neighbourhood
1001       2000  1985       42
1001       2001  1985       42 ← repeated
1001       ...   1985       ... One typo = many broken rows
```

With separation:

```
individuals: person_id=1001, birth_year=1985 ← once
residences:  person_id=1001, year=2000, neighbourhood=42
residences:  person_id=1001, year=2001, neighbourhood=67
Fix birth_year in one place → done
```

Municipality Schema — The Five Tables

All relationships here are one-to-many



PK

Primary key

FK

Foreign key · No many-to-many here (all relationships are one-to-many)

The Payoff — Preparing for the Research Question in SQL

All five tables in one query — impossible with flat files

```
-- Employment rate in early adulthood (age 25-30)
-- by poverty level of neighbourhood during childhood (age 10-15)

SELECT
  CASE
    WHEN n.poverty_rate > 0.25 THEN 'high poverty'
    WHEN n.poverty_rate > 0.10 THEN 'medium poverty'
    ELSE 'low poverty'
  END AS childhood_neighbourhood,
  ROUND(AVG(e.employed), 3) AS employment_rate,
  COUNT(DISTINCT e.person_id) AS n_individuals
FROM employment e
JOIN individuals i ON e.person_id = i.person_id
JOIN residences r ON e.person_id = r.person_id
JOIN neighbourhoods n ON r.neighbourhood_id = n.neighbourhood_id
WHERE (e.year - i.birth_year) BETWEEN 25 AND 30 -- adult years
AND (r.year - i.birth_year) BETWEEN 10 AND 15 -- childhood years
GROUP BY childhood_neighbourhood
ORDER BY employment_rate DESC
```

childhood_neighbourhood	employment_rate	n_individuals
high poverty	0.97	1423
high poverty	0.84	1532
low poverty	0.82	5732
medium poverty	0.79	972
low poverty	0.73	12133

Without the relational structure, answering this question requires manually merging four separate files every time — and re-merging whenever any source file is updated. With the database, it is one query.

What If the Relationship Is Many-to-Many?

The municipality data only had one-to-many. Now, something new!

Example 1 — one-to-many

One individual → many annual employment rows
One neighbourhood → many annual residence rows

Solution: put the foreign key on the 'many' side.

employment table gets person_id FK.
residences table gets person_id FK.



but now...

Example 2 — many-to-many

An article has many authors.
An author has written many articles.

You cannot put author_id on the articles table, as there would be multiple authors.

You cannot put article_id on the authors table, as there would be multiple articles.

Neither side can hold the foreign key.
You need a third (junction) table!

This third table is called a junction table, which is what Example 2 is about.

Example 2 — Academic Literature Database

Directly relevant to computational social science research

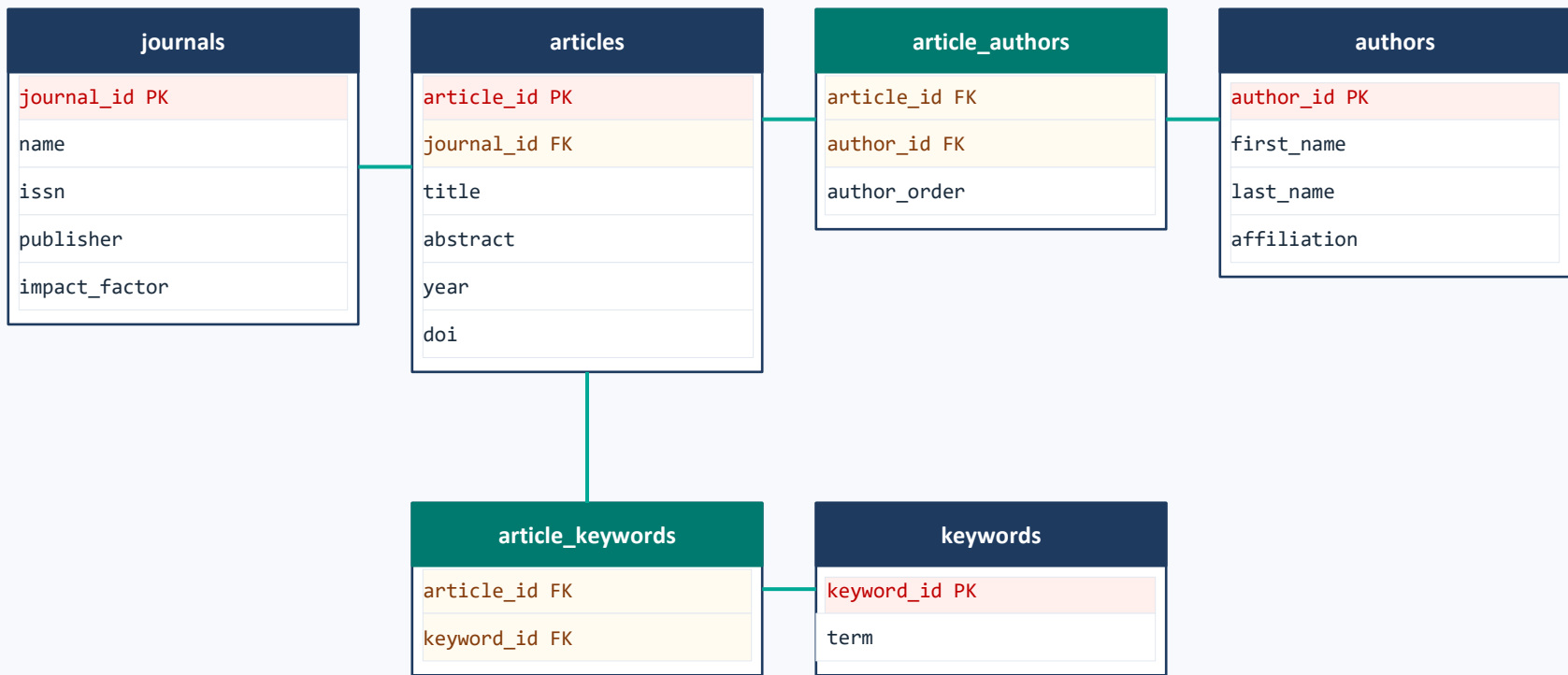
You have scraped 500 articles from three sociology journals. Each article has a title, abstract, year, DOI, and citation count. Each article has one or more authors with affiliations. Each article appears in one journal. Each article has keywords.

Identifying the entities and their relationships:

1:N	One-to-many <i>One journal → many articles</i>	journal_id FK on articles
N:M	Many-to-many <i>Many articles ↔ many authors</i>	Junction table article_authors(article_id, author_id)
N:M	Many-to-many <i>Many articles ↔ many keywords</i>	Junction table article_keywords(article_id, keyword_id)

ER Diagram — Academic Articles Database

Junction tables bridge the many-to-many relationships



PK

Primary key

FK

Foreign key

J

Junction table (many-to-many bridge)



Break!

2

Creating Tables in SQLite from R

Full workflow · CREATE TABLE · Constraints

The Full R Workflow — Creating a Database From Scratch

Everything you need, in order, using the municipality example

```
library(DBI)
library(RSQLite)

# 1. Create or open the database file (will be a non-existent file if you create one from scratch!)
con <- dbConnect(SQLite(), "data/municipality.sqlite")

# 2. Enable foreign key enforcement (run every session)
dbExecute(con, "PRAGMA foreign_keys = ON")

# 3. Create tables – parent before child
dbExecute(con, "
CREATE TABLE IF NOT EXISTS municipalities (
  municipality_id INTEGER PRIMARY KEY,
  name             TEXT NOT NULL,
  region           TEXT
)
")

# verify
dbListTables(con)
```

Key points: dbConnect creates the file if it does not exist, opens it if it does. · IF NOT EXISTS makes CREATE TABLE safe to re-run. · Always run PRAGMA foreign_keys = ON before writing data.

CREATE TABLE — Child Tables and FOREIGN KEY Constraints

Creating tables that reference other tables

```
# — neighbourhoods: references municipalities —————
dbExecute(con, "
CREATE TABLE IF NOT EXISTS neighbourhoods (
  neighbourhood_id INTEGER PRIMARY KEY,
  municipality_id  INTEGER NOT NULL,
  name             TEXT,
  poverty_rate     REAL,
  urban_rural      TEXT,
  FOREIGN KEY (municipality_id) REFERENCES municipalities(municipality_id)
)
")

# — residences: references both individuals and neighbourhoods —————
dbExecute(con, "
CREATE TABLE IF NOT EXISTS residences (
  person_id        INTEGER NOT NULL,
  year             INTEGER NOT NULL,
  neighbourhood_id INTEGER NOT NULL,
  housing_tenure   TEXT,
  FOREIGN KEY (person_id) REFERENCES individuals(person_id),
  FOREIGN KEY (neighbourhood_id) REFERENCES neighbourhoods(neighbourhood_id)
)
")
```

Always create parent tables first. You cannot create neighbourhoods before municipalities because the FOREIGN KEY reference would fail.

CREATE TABLE — Child Tables and FOREIGN KEY Constraints

Creating tables that reference other tables

```
# — employment: references individuals
dbExecute(con, "
CREATE TABLE IF NOT EXISTS employment (
  person_id    INTEGER NOT NULL,
  year         INTEGER NOT NULL,
  employed     INTEGER,      -- 1 = yes, 0 = no
  sector       TEXT,
  annual_income REAL,
  FOREIGN KEY (person_id) REFERENCES individuals(person_id)
)
")
```

Always create parent tables first. You cannot create neighbourhoods before municipalities because the FOREIGN KEY reference would fail.

Constraints: Rules that the Database Enforces for You

Let the database do the data quality work

Constraint	What it does	Example
PRIMARY KEY	Uniquely identifies each row. No duplicates, no NULLs. SQLite auto-increments INTEGER primary keys.	<code>person_id INTEGER PRIMARY KEY</code>
NOT NULL	The field must always have a value. An INSERT without it will fail.	<code>birth_year INTEGER NOT NULL</code>
UNIQUE	No two rows can share this value. A table can have multiple UNIQUE columns (unlike PK).	<code>doi TEXT UNIQUE</code>
FOREIGN KEY	References a PK in another table. Requires PRAGMA foreign_keys = ON to be enforced in SQLite.	<code>FOREIGN KEY (municipality_id) REFERENCES municipalities(municipality_id)</code>
DEFAULT	Fallback value if none is given at INSERT time.	<code>employed INTEGER DEFAULT 0</code>

SQLite data types: INTEGER (IDs, years, 0/1 flags) REAL (decimal numbers, rates) TEXT (names, DOIs, categories) BLOB (raw binary — rarely needed).

Booleans → INTEGER (0/1) · Dates → TEXT ('2020-04-15') · Identifiers like DOI → TEXT even if they look numeric

INSERT, UPDATE, DELETE — Modifying Data

The write operations — executed immediately, no undo

```
# INSERT: add new rows. While scraping, collecting: do this!
dbExecute(con, "
  INSERT INTO municipalities (name, region)
  VALUES ('Linköping', 'Östergötland')
")
-- municipality_id is auto-assigned — don't include it in the INSERT

# UPDATE: change existing rows — ALWAYS include WHERE
dbExecute(con, "
  UPDATE neighbourhoods
  SET poverty_rate = 0.18
  WHERE neighbourhood_id = 42
")
-- Without WHERE, every row is updated

# DELETE: remove rows — ALWAYS include WHERE
dbExecute(con, "
  DELETE FROM employment
  WHERE person_id = 1001 AND year = 2005
")
-- Without WHERE, every row is deleted
```

Remember

- All three execute immediately on the .sqlite file. No undo.
- Make a backup before running UPDATE or DELETE on real data.
- Forgetting WHERE in UPDATE or DELETE affects every row.
- PRAGMA foreign_keys = ON prevents deleting a municipality that still has neighbourhoods referencing it.

SELECT, INSERT, UPDATE, DELETE together are called CRUD — Create, Read, Update, Delete. These four operations cover everything you do to a database.

Loading Real Data — dbWriteTable from R

Populating a table from a data frame in one call

```
# Let's say that you have cleaned your data in R as a data frame.
# You can then write it directly to the database – no INSERT loops needed.

dbWriteTable(con, "individuals", individuals_df, append = TRUE)
#   append = TRUE    → add rows to an existing table
#   overwrite = TRUE → drop and recreate the table (don't use both)

# Column names in individuals_df must match the table column names.
# Check with:
names(individuals_df)
dbListFields(con, "individuals")

# For tables with two foreign keys (like residences):
# Build the data frame first with both FK columns present, then write.
residences_df <- raw_address_data |>
  select(person_id, year, neighbourhood_id, housing_tenure)

dbWriteTable(con, "residences", residences_df, append = TRUE)

# Verify that the tables in the database have the same number of lines:
dbGetQuery(con, "SELECT COUNT(*) FROM individuals")
dbGetQuery(con, "SELECT COUNT(*) FROM residences")
```

Tip: always write parent tables before child tables. You cannot insert a residence with municipality_id = 5 if municipality 5 does not yet exist.

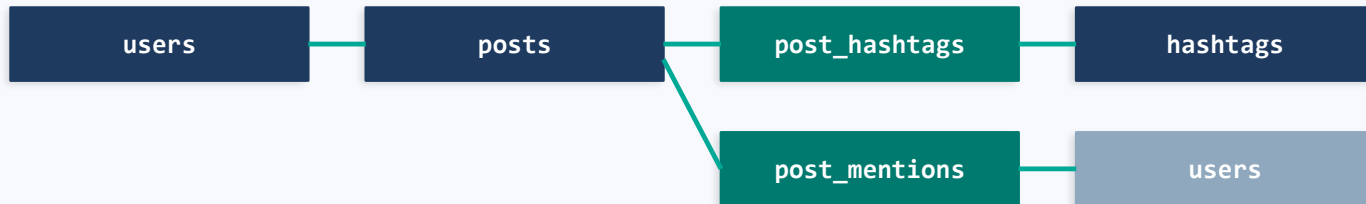
3

Design patterns

Three schemas from Computational Social Science

Pattern A — Social Media Data

Users, posts, hashtags, and a self-referential relationship



users → posts (one-to-many)

A user has many posts. Add `user_id` as a FK on posts.

posts ↔ hashtags (many-to-many)

A post has many hashtags; a hashtag appears in many posts. Bridge with junction table: `post_hashtags(post_id, hashtag_id)`.

post_mentions is self-referential

A mention is a relationship between two users: both live in the users table. `post_mentions(post_id, mentioned_user_id)` where `mentioned_user_id → users.user_id`.

Pattern B — Politicians and Speeches

When a relationship itself has attributes



Key insight: **party_memberships** needs its own attributes

A politician can change party over time. Storing just `party_id` on `politicians` loses that history. `party_memberships(politician_id, party_id, start_date, end_date)` captures the full timeline. When a relationship has its own attributes, it deserves its own table.

politicians → speeches (one-to-many)

One politician gives many speeches. One speech 'has' one politician. So: add `politician_id` as FK on `speeches`.

speeches could also link to topics

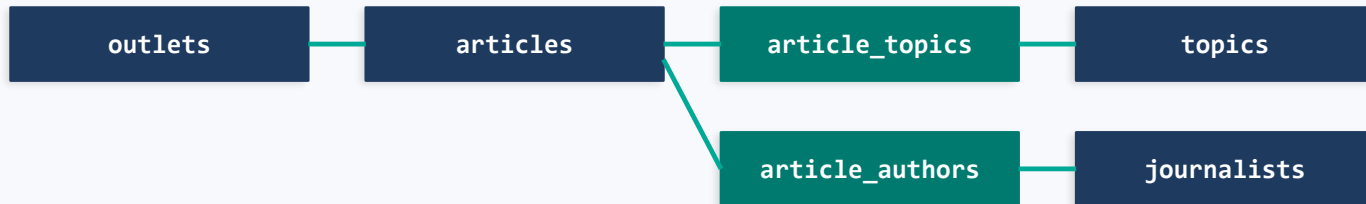
Add a `speech_topics` junction if each speech can be tagged with multiple topics (the same pattern as *article_keywords*). 'speeches' can of course contain a field for the filepath (i.e. where you have stored a text file with the actual speech)

Useful queries

"Which party did politician X belong to in 2018?" · "Which parties have given speeches about topic Y?"

Pattern C — News Articles and Media Outlets

The structure of a journalism or media analysis project



outlets → articles (one-to-many)

An article is published by one outlet. An outlet publishes many articles. Add `outlet_id` as FK on articles.

articles ↔ topics (many-to-many)

An article can cover multiple topics; a topic spans many articles. Use `article_topics(article_id, topic_id)`, i.e. junction table, to bridge.

articles ↔ journalists (many-to-many)

A piece can have multiple by-lines (journalists). A journalist writes many pieces. Use `article_authors(article_id, journalist_id, author_order)`.

Connecting to other sources

outlets can link to owner entities (media groups), enabling cross-source analysis.

Useful queries

"Which outlet covers topic X most?" · "Which journalists co-author most often?" · "How has topic coverage changed over time?"

The General Message — and When NOT to Use a Database

Design is a social science decision, not a technical one

Before you scrape, design your schema.

It is far harder to restructure a populated database than to design it correctly from the start.

The schema is a theory of what entities exist in your domain and how they relate.

That is a social science question.

Asking it forces you to think clearly about your research design before touching any code.

While scraping

INSERT what you scrape into the database tables you have designed!

When NOT to use a relational DB:

- Your data is genuinely one flat table: a CSV is fine
- Your data is unstructured text: consider a document store (MongoDB)
- You need to store graph relationships: consider a graph database
- Very high write speed at massive scale: specialised systems exist

When relational databases shine: you have multiple entity types, the data is updated over time, you need to query across entities flexibly, and multiple people or scripts write to the same data.

4

Lab

Design, Build, Populate, and Query

Lab 2 — Build Your Own Database

Work in pairs · choose one scenario · three stages

Stage 1

Design
(45 min)

- Identify entities and attributes
- Draw the ER diagram on paper
- Decide on primary and foreign keys
- Peer-review with another pair

Stage 2

Build
(45 min)

- Write CREATE TABLE statements in R
- Use correct types and constraints
- Populate ≥ 2 tables with realistic data
- Run one INSERT, UPDATE, DELETE

Stage 3

Query & reflect
(30 min)

- Write ≥ 2 JOIN queries that need the schema
- What questions require multiple tables?
- Write: one design decision and why
- What would break with a flat CSV?

End result: a working .sqlite file

Choose Your Scenario

Pick one : all involve real CSS research data situations

Scenario A

Parliamentary Speeches

Speeches from a national parliament over 10 years. Speakers, dates, durations, topics, text. Speakers belong to parties — and can change party. Design the database.

Scenario B

Social Media Influencers

500 Instagram accounts with follower counts tracked weekly, posted images with captions, and hashtags. Must answer: which hashtags co-occur? How has follower count changed?

Scenario C

Survey + Admin Data

Survey of 300 respondents on political attitudes, linked to their municipality, plus municipality-level statistics (population, unemployment, urban/rural). Three sources, one database.

Scenario D

Citation Network

1,000 articles from three sociology journals with titles, abstracts, authors, years, and lists of cited DOIs. Must answer: which articles are cited most? Which authors cite each other?

You may also propose your own scenario! Preferably having ≥ 3 entity types and at least one many-to-many relationship.

Stage 2 — Build: Code Skeleton to Start From

Fill in the ... with your own table names and columns

```
library(DBI)
library(RSQLite)

# Create a new database file for your project
con <- dbConnect(SQLite(), "data/my_project.sqlite")
dbExecute(con, "PRAGMA foreign_keys = ON")

# Write your CREATE TABLE statements here
# Remember: parent tables before child tables
dbExecute(con, "
  CREATE TABLE IF NOT EXISTS ... (
    ...
  )
")

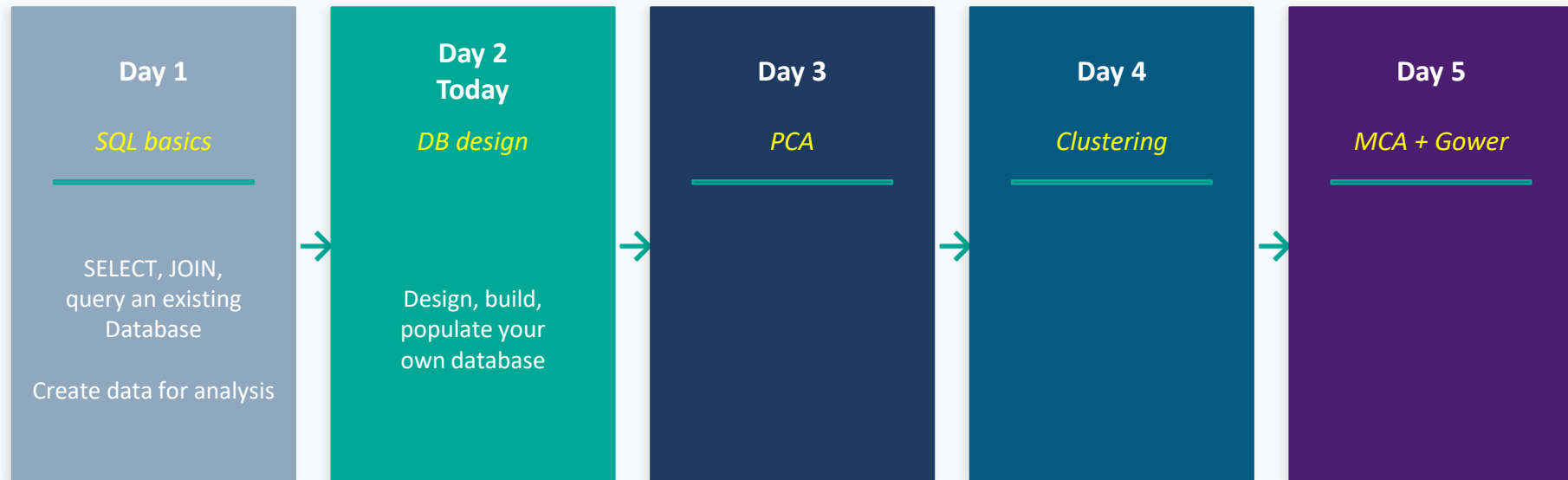
# Populate at least two tables – invent realistic data
my_df <- data.frame(name = c("..."), ...)
dbWriteTable(con, "...", my_df, append = TRUE)

# Verify structure and content
dbGetQuery(con, "PRAGMA table_info(...)")
dbGetQuery(con, "SELECT * FROM ... LIMIT 5")

# Practice CRUD: one INSERT, one UPDATE, one DELETE
dbExecute(con, "INSERT INTO ... VALUES (...)")
dbExecute(con, "UPDATE ... SET ... WHERE ...")
dbExecute(con, "DELETE FROM ... WHERE ...")
```

The Thread Connecting All Five Days

SQL is infrastructure, and the EDA methods are what run on it



Next session: Day 3 — we take the Spotify genre data and run PCA on it.